



RAGAS Metrics Unpacked

RAGAS isn't one score. It's a set of metrics, each measuring a different failure point. Here are the three I used most on my compliance agent. And what each one actually told me.



01

Faithfulness

Faithfulness asks: is the answer supported by the retrieved context? My agent was occasionally adding correct-sounding information that wasn't in the retrieved chunks. The LLM was filling gaps from training data. In banking, that's not acceptable. Faithfulness caught it. I tightened the prompt.





02

Context Relevancy

Context Relevancy asks: are the retrieved chunks actually useful for answering the question? This is where my pipeline was weakest. I was retrieving chunks from the right articles — but wrong sections. The retrieval looked correct. The quality wasn't. This metric showed me that.





03

Answer Relevancy

Answer Relevancy asks: does the answer actually address the question asked? My agent was sometimes answering adjacent questions — technically accurate, but not what was asked. Regulatory text is dense. It's easy to drift. This metric kept the answers focused.



04

How I Used Them Together

- Faithfulness → model problem. Tighten the prompt or retrieval grounding.
- Context Relevancy → retrieval problem. Fix chunking or embedding strategy.
- Answer Relevancy → generation problem. Revisit how the question is passed to the LLM.

Three metrics. Three different places to fix.



05

More Metrics in RAGAS

These three aren't the only ones.

RAGAS also includes:

- Context Precision
- Context Recall
- Answer Correctness
- Answer Similarity

Each one targets a different failure point. The three I covered were the most relevant to my use case.



06

Custom Metrics

RAGAS also lets you define your own metrics. You describe what "good" looks like for your domain. It uses an LLM as a judge to score against that criteria. For compliance, I could define exactly what a correct regulatory answer looks like. That's powerful.





07

The Honest Takeaway

RAGAS didn't make my agent perfect. It made my debugging precise. Instead of changing things and hoping, I had scores to move. If you're building a RAG-heavy agent — don't skip evaluation. Measure early. Measure often. Know where your pipeline is actually breaking.

